

Individual Assignments

Matrix Multiplication

BIG DATA · Academic year 2026-2027

Grado en Ciencia e Ingeniería de Datos

Universidad de Las Palmas de Gran Canaria

Purpose

You will conduct four individual empirical studies of matrix multiplication. Each study must begin with a focused research question, use controlled measurements, and draw conclusions supported by the observed results. The goal is to understand performance and scalability and to communicate a reproducible investigation.

Benchmarking measures and compares implementations under specified conditions. These assignments examine execution time and memory use, then extend the investigation to optimisation, sparse representations, parallel execution and distributed execution.

Four stages and submission windows

Assignment	Submission week	2026 date window
1. Basic implementations in Python, Java and C	Week 6	12-16 October
2. Optimised approaches and sparse matrices	Week 8	26-30 October
3. Parallel multiplication; optional vectorisation	Week 10	9-13 November
4. Distributed multiplication	Week 12	23-27 November

These are submission windows from the course plan, not exact deadlines.

How the assignments connect

Reuse validated implementations and experimental scripts across stages. Each submission must remain understandable on its own and identify the earlier implementation or dataset it reuses. The number of required methods is specified in each assignment; there is no blanket three-method requirement.

Common experimental protocol

Applies to all four assignments

1. Define the question and comparison

State what you will investigate, the factor you will change, and the outcome you will measure. For example: "How does matrix size affect runtime and memory use for the three basic implementations?" Define n as the side length of an $n \times n$ square matrix. Distinguish hypotheses or predictions from results.

2. Control the inputs and environment

- Use the same logical input matrices across methods for each comparison. Record the random seed, generator, value range, dimensions and numeric type. Cross-language random generators may differ, so a shared input file or a verified common generator is preferable.
- Document CPU model, physical cores and logical processors, RAM, operating system, language and library versions, compiler flags, and relevant runtime settings. Record thread counts, including threads that numerical libraries create internally.
- Keep the algorithm and arithmetic comparable when the comparison requires it. Report unavoidable differences in storage or runtime behaviour. Do not treat different machines as an isolated comparison of programming languages.

3. Define what the timer includes

State whether you measure the multiplication kernel or the whole application. For kernel measurements, generate inputs outside the timed region. Explain whether allocation and output initialisation are included. For end-to-end measurements, specify data loading, setup, transfers and result collection. Never combine different timing boundaries in one unqualified comparison.

4. Repeat and summarise

- Use a benchmarking tool or a documented timing harness suitable for the implementation. Use a monotonic timer and ensure that the computation actually executes and that its result is consumed or checked.
- Use warm-up runs when runtime compilation, caching or framework startup can affect the measurement. Exclude warm-up runs from reported samples and record their number.
- As a recommended starting protocol, collect at least five measured repetitions per configuration. Justify fewer repetitions for costly runs and report the resulting uncertainty. Very short tasks may require repeated batches with a clearly reported time per multiplication.
- Keep every raw measurement. Report a central measure, preferably the median, and variability, such as the interquartile range. Include units and the number of repetitions. Explain exclusions rather than removing inconvenient observations.

Correctness, resources and reproducibility

Validate before benchmarking

Test small matrices with known answers, including identity and zero matrices. Compare results against a trusted reference implementation. A reference library used for checking correctness does not automatically count as a required student implementation.

For floating-point values, define absolute and relative tolerances and explain why they are appropriate. Different evaluation orders can produce small numerical differences. For integer arithmetic, consider overflow and document the value range.

Choose feasible sizes

Begin with small correctness tests, then increase n using a declared size sequence. Values such as 10, 100 and 1,000 are starting examples, not compulsory performance targets. Add sizes that reveal a trend and stop when the declared time or memory budget is reached.

A dense $n \times n$ matrix with 64-bit values requires $8n^2$ bytes for its elements. At $n = 10,000$, three such matrices require about 2.4 GB (decimal), before temporary buffers, object overhead or runtime memory. This estimate is a planning aid, not a requirement to execute that size.

Define a maximum runtime and a memory budget before large tests. State the largest tested size that meets those limits, separately from the largest size attempted. Report timeouts and memory failures as results with their limits; do not assign them an invented execution time.

Memory and resource measurements

Measure execution time and memory in every assignment. Identify the memory metric and tool: for example, peak process resident memory, managed heap use, or an estimate of array storage. These quantities differ and should not be compared as if they were identical. Report measurement limitations. CPU utilisation is optional unless it is needed to explain a result.

Make the study reproducible

- Provide source code, a README with exact build and run commands, dependency versions, and configuration files. Explain how to regenerate datasets or obtain the shared inputs.
- Provide raw measurements as CSV or another documented machine-readable format, plus scripts or notebooks that generate the tables and plots.
- Label every configuration with matrix size, method, numeric type, repetition and relevant thread or worker count. For sparse tests, include density and representation.

Keep each study small enough to complete and inspect carefully. A defensible controlled comparison is the core requirement; an inaccessible large configuration does not improve the quality of the investigation.

Assignment 1

Basic Matrix Multiplication in Different Languages

Research objective

Compare a basic dense matrix multiplication algorithm implemented in Python, Java and C. Investigate how matrix size affects runtime and memory under the documented execution conditions.

Required implementations and comparisons

- Implement the straightforward triple-loop algorithm in all three languages. Each implementation must use $O(n^3)$ arithmetic work for $n \times n$ inputs.
- Use a comparable loop order and numeric type. Describe each storage layout and any unavoidable language-specific differences. Do not replace the required loops with an optimised matrix multiplication library call.
- Test a feasible increasing sequence of matrix sizes using the common protocol. For each size, compare all three implementations wherever runs complete within the declared budget.
- Document the compiler and optimisation flags for C, JVM and warm-up settings for Java, and Python implementation and version. Explain which settings remain fixed.

Required evidence

Measure	What to report
Time	Repeated measurements, median and variability for every tested configuration
Memory	Named memory metric and measurement tool for each implementation
Correctness	Test cases, reference results and any numerical tolerance
Limits	Largest tested feasible size and any timeouts or memory failures

Analysis expected

Explain the observed growth and discuss interpreter, runtime and memory-layout effects where your evidence supports them. Compare the trend with the cubic algorithmic model while recognising constants and system effects.

Scope conclusions to the tested implementations, settings, inputs and hardware. Avoid a general claim that one programming language is universally faster. Language-specific optimisations can form additional experiments, but must remain separate from the controlled basic comparison.

Submission

Week 6: 12-16 October 2026. Submit the research report, code and reproducibility materials described on page 8. Exact closing day and time: to be confirmed.

Assignment 2

Optimised Matrix Multiplication Approaches and Sparse Matrices

Research objective

Investigate two questions: how selected optimisations affect dense multiplication, and under which tested sparsity conditions a sparse representation offers an advantage.

Part A: dense optimisation

- Use one validated basic implementation as the baseline. Implement at least two distinct optimised versions, giving at least three dense methods in total.
- Possible techniques include cache blocking, loop reordering, loop unrolling, or an algorithmic approach such as Strassen. Identify the actual change and explain its expected effect. These are examples, not compulsory choices.
- Compare the baseline and both optimised methods with the same input matrices, numeric type and timing boundary. Keep the comparison sequential unless a parallel variant is explicitly identified and analysed separately.

Part B: sparse matrices

- Implement a sparse multiplication approach. State the storage format, such as CSR, COO or another documented representation, and identify your own code and any library functionality.
- Test multiple declared sparsity levels. Define sparsity as the percentage of zero entries and density as the percentage of nonzero entries. Record the input generation process, nonzero distribution and seeds.
- For matched logical inputs, compare the sparse approach with a dense method. Specify whether conversion costs are excluded, included, or measured separately. Report input and output nonzero counts because the product can become denser.

Required evidence and analysis

Report time and memory with repeated measurements, the largest feasible tested size under the declared budget, and observed bottlenecks. Discuss the effects of optimisation separately from the effects of sparse storage and input structure.

Identify where sparse multiplication helps in your experiments and where its indexing, conversion or output costs reduce the advantage. Do not claim that a percentage of zeros alone determines performance in every case.

Submission

Week 8: 26-30 October 2026. Submit the research report, implementations, input configurations and raw measurements. Exact closing day and time: to be confirmed.

Assignment 3

Parallel Matrix Multiplication and Optional Vectorisation

Research objective

Investigate how thread count affects the performance of a shared-memory matrix multiplication implementation, and explain when parallel execution helps.

Required implementations and comparisons

- Provide an equivalent sequential baseline and a parallel implementation using threads or a framework such as OpenMP. Comparing the parallel method with its sequential counterpart is mandatory.
- Use the same algorithm, numeric type, storage layout and input matrices where possible. Document differences. A comparison with a much slower basic algorithm can be added, but cannot replace the equivalent baseline.
- Test several thread counts supported by the available hardware, including one thread. Record physical cores, logical processors and any affinity settings. Identify oversubscription if you test more threads than hardware execution contexts.
- Apply the common timing and repetition protocol. Check correctness and safe access to shared data. Control internal library threads so that the reported thread count describes the actual configuration.

Metrics

Let $T_{\text{seq}}(n)$ be the sequential baseline time and $T_p(n)$ the parallel time for the same input size with p threads. Define **speedup** $S(p) = T_{\text{seq}}(n) / T_p(n)$ and **efficiency** $E(p) = S(p) / p$. State which summary statistic you use for these times.

Report runtime, variability, memory, speedup and efficiency. Also report the parallel implementation with one thread to reveal framework or implementation overhead. Explain any unusual or superlinear speedup using evidence rather than assuming a measurement is correct.

Optional vectorisation extension

You may implement or evaluate SIMD vectorisation. Describe compiler options or explicit instructions and the evidence that vectorisation occurs. Compare vectorised and non-vectorised versions under controlled conditions. Clearly separate vectorisation effects from changes in thread count.

If you do not undertake this extension, no vectorisation result is required.

Submission

Week 10: 9-13 November 2026. Submit the report, code, thread configurations and measurements. Exact closing day and time: to be confirmed.

Assignment 4

Distributed Matrix Multiplication

Research objective

Investigate how a distributed matrix multiplication implementation scales with workers or nodes, considering computation, communication and framework overhead.

Required implementation and comparisons

- Implement matrix multiplication using a distributed computing framework. Explain the partitioning of matrices, placement of computation and collection or storage of the result.
- For feasible common sizes, compare the distributed implementation with a validated basic sequential version and the parallel version. Document any hardware, algorithm or timing differences that prevent an isolated comparison.
- Test multiple worker or node configurations where resources allow. Distinguish workers, processes and physical machines.
- Use sufficiently large feasible inputs to investigate scaling. Inputs larger than one machine's memory are an extension when infrastructure permits, not a universal requirement.

Scaling and overhead

For **strong scaling**, keep the total problem size fixed while increasing resources. If you also study **weak scaling**, explain how you increase the workload with resources; for cubic dense multiplication, simply keeping n per worker fixed is not a sufficient definition of equal work.

Report end-to-end time with an explicit boundary. Where supported by the framework, also report computation, setup, transfer and result-collection costs. Explain overlapping activities and avoid presenting overlapping timings as additive components.

Report resource counts, memory per worker or node where measurable, data-transfer measurements where available, and the tools used. If network costs cannot be isolated, state this limitation and analyse the available observations without inventing measurements.

Local execution limitation

A local execution with multiple workers may be used only if the instructor permits it. It can investigate framework behaviour and process overhead, but does not demonstrate inter-machine network performance or multi-machine memory capacity. Label the environment clearly.

Submission

Week 12: 23-27 November 2026. Submit the report, code, framework configuration and reproducibility materials. Exact closing day and time: to be confirmed.

Research report and submission package

A common structure for every report

Section	What it must explain
Title and short abstract	The question, method, main result and scope of the conclusion
Introduction and objective	The motivation, a focused research question and any hypothesis
Background and references	Relevant algorithms or techniques and sources that support the explanation
Methodology	Implementations, inputs, environment, correctness checks, measurement protocol and budgets
Results	Clearly labelled tables and plots, units, repetitions and variability
Discussion	An answer to the question, plausible explanations supported by evidence, and limitations
Conclusions	What the study establishes within its tested scope and what remains unresolved
References and materials	Full citations and the location or description of reproducibility materials

Write the report in English. Keep it concise and sufficiently detailed to reproduce the experiment. Use this as a course research report; a novel algorithm or publication-level contribution is not required.

Submission contents

- A PDF report with your name, assignment number and research title.
- Source code, a README, dependency and environment details, and exact build and run instructions.
- Raw results, input generation or acquisition instructions, and scripts that reproduce the analysis and figures. Include reused materials or identify their exact version.

The submission channel and packaging convention will be announced. Suggested names are Assignment01_Surname_Report.pdf and Assignment01_Surname_Materials.zip; a repository such as GitHub is mandatory.

Figures, citations and oral explanation

Label axes and units, name configurations, and explain what each figure shows. Cite borrowed ideas, code, text and figures. Use one consistent reference style, such as IEEE, and verify that each source exists and supports the claim.

If an oral presentation is required, explain the research question, the design that makes the comparison fair, the most informative results and the limitations. Be ready to explain your code and reproduce or interpret a selected experiment. Duration and presentation requirements remain to be confirmed.

Assessment and notes

Assessment criteria

Criterion	Evidence of a satisfactory study
Correctness	Validated results and clearly documented numerical assumptions
Experimental design	Relevant question, fair baselines, controlled inputs and a justified protocol
Analysis	Interpretation of time, memory and scaling with variability and limitations
Reproducibility	Executable code, configuration details, raw data and analysis scripts
Communication	Clear English report, readable figures, accurate references and defensible conclusions

Criterion weights, assignment weights and the treatment of optional extensions must follow the course assessment scheme. This proposal does not assign numerical weights or change the published grading scheme.

Proposed generative AI policy

AI tools may support explanation, planning, debugging or language revision. Students must disclose the tool, purpose and affected parts of the work. They remain responsible for understanding the code, validating measurements, verifying references and defending the conclusions. Generated content is not evidence. Fabricated results or references are unacceptable.

[Course context](#)